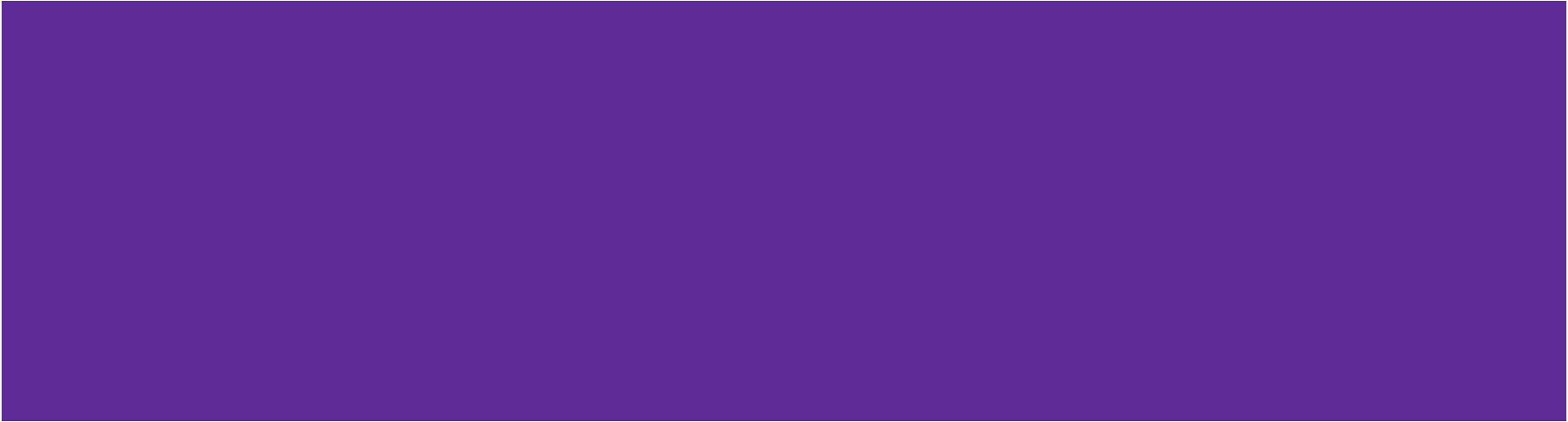


DISTRIBUTED AND CLOUD COMPUTING

LAB 4: SERVICES & API ARCHITECTURES

(Module: Services & API Architectures)



Local Program ➡ Web Service

- Local Program ➡ Modular Service
 - Task Orientation + Reusability**: group functions/procedures as a **service**.
 - Independent Development**: Separate client and service code - **Modularization**.

```

my_program.py X
1 # Local Procedure: Greet the user with provided user name and institution.
2 def greet_with_info(username, institution):
3     return f'Hello {username} from {institution}!'
4
5 # Local Procedure: Multiply two numbers.
6 def mult(xin, yin):
7     return xin * yin
8
9 # Client to use the Procedure.
10 if __name__ == '__main__':
11     print(greet_with_info(username='Peter', institution='SUSTech'))
12     print(mult(xin=3.5, yin=5))
13
    
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```

(dncc) (base) root@RAINBOW:~/rainbow/asialab/dncc/local_service# python my_program.py
Hello Peter from SUSTech!
17.5
    
```

```

my_service.py X
1 my_service.py > AssistantService > greet_with_info
2 # A Modular Assistant Service.
3 class AssistantService:
4     def __init__(self) -> None:
5         # Add some properties.
6         pass
7
8     # Greet the user with provided user name and institution.
9     def greet_with_info(self, username, institution):
10         return f'Hello {username} from {institution}!'
11
12     # Multiply two numbers.
13     def mult(self, xin, yin):
14         return xin * yin
    
```

What if service is no longer local?

```

my_client.py X
1 my_client.py > ...
2 from my_service import AssistantService
3
4 # Client to use the Procedure
5 if __name__ == '__main__':
6     svc = AssistantService()
7     print(svc.greet_with_info(username='Peter', institution='SUSTech'))
8     print(svc.mult(xin=3.5, yin=5))
    
```

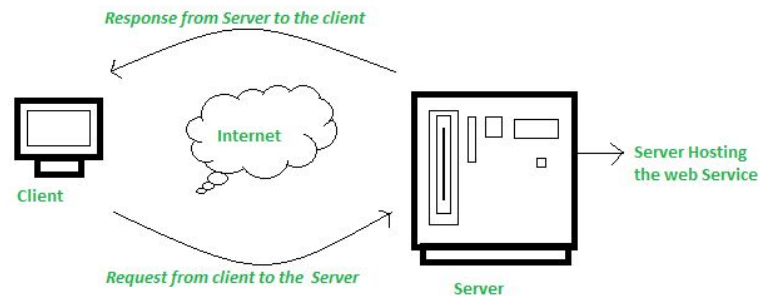
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```

(dncc) (base) root@RAINBOW:~/rainbow/asialab/dncc/local_service# python my_client.py
Hello Peter from SUSTech!
17.5
    
```

Local Program ➡ Web Service

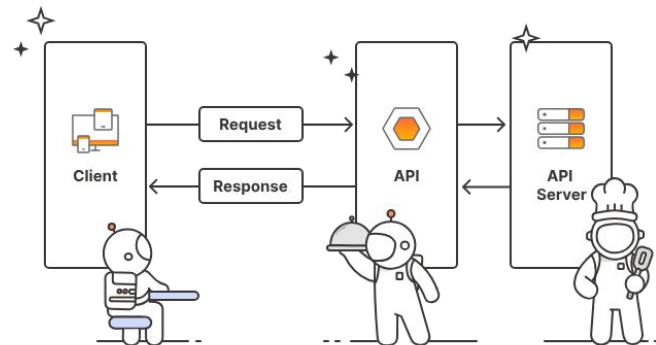
- Local Program ➡ Modular Service
 - a. Task Orientation + Reusability: group functions/procedures as a **service**.
 - b. Independent Development: Separate client and service code - **Modularization**.
- Local Service ➡ Web Service
 - a. **Web Service**: “a service offered by an electronic device to another electronic device, communicating with each other via the Internet”.
 - b. Making services web-ready: utilize web standards.
 - Network Protocols (e.g., HTTP, HTTPS)
 - Data Formats (e.g., XML, JSON)
- Web Services specify communication standards and interfaces, making them **APIs**.



Application Programming Interface (API)

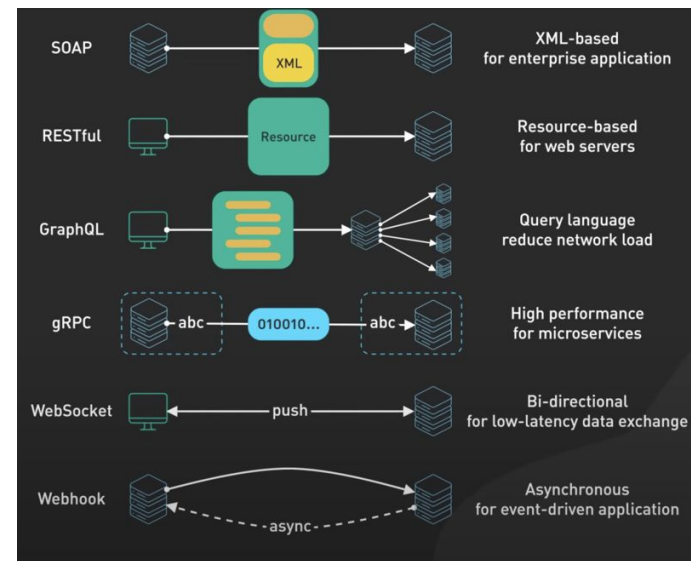
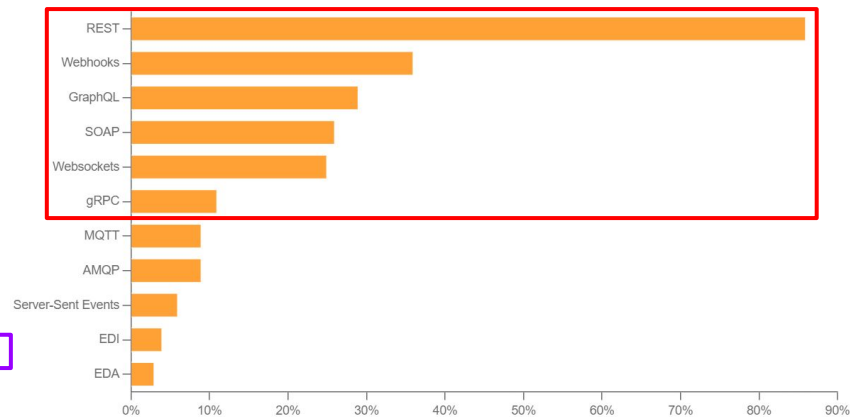
- “APIs are mechanisms that enable two software components to communicate with each other using a set of definitions and protocols.”
- Examples:
 - a. [GitHub REST API](#) - build scripts/apps for automation and extension
 - b. [OpenAI API](#) (RESTful API) - programmatically interact with LLMs like ChatGPT
 - c. [Slack Web API](#) (gRPC) - query and manage Slack messages, channels, workspaces
 - d. [Netflix Backend-to-backend Communication](#) (gRPC)
 - e. ...

- There are different ways to design and build APIs, each tailored to different purposes and scenarios. These designs are referred to as “**API Architecture Patterns**”.



API Architecture Patterns

- **SOAP:**
 - uses XML messages + a predefined contract
 - strict, complex, and verbose
- **RESTful API:** **Currently the most popular architecture for Web Services!**
 - manages resources via common web standards
 - uses HTTP methods to operate on JSON/XML
 - simple, fast, and flexible
- **GraphQL:**
 - queries exactly needed within a single request
 - suitable for complex data requirements
- **RPC:**
 - accesses remote services as if they were local
 - abstracts the complexity of network communication
- **WebSocket:**
 - enables fast, bidirectional, and persistent connections
 - benefits live chat apps and real-time gaming
- **Webhook:**
 - supports asynchronous & event-driven notifications



<https://www.postman.com/what-is-an-api/#api-architectural-styles>

https://dev.to/kanani_nirav/top-6-most-popular-api-architecture-styles-you-need-to-know-with-pros-cons-and-use-cases-564j

TASK: RESTful API - Hello World with Python Flask

Implement a simple RESTful API server using Python Flask.

> Reference codebase: [rest_hello_world](#)

1. Set up Python ([uv](#) is recommended).
2. Install Python dependencies into a uv environment via:
 - `uv sync`
3. Run the API server via:
 - `uv run python server.py`
4. In another terminal, test the API with HTTP requests.

```
21 @app.route('/', methods=['GET'])
22 def greet():
23     return {'message': 'Hello World!'}, 200
24
25 @app.route('/chat/<username>', methods=['GET'])
26 def greet_with_info(username): # retrieve username from URL path
27     # retrieve institution from URL query
28     institution = request.args.get('institution', None)
29     institution_segment = f' from {institution}' if institution else ''
30     msg = f'Hello {username}{institution_segment}!'
31     return {'message': msg}, 200
32
33 @app.route('/calculator/mult', methods=['POST'])
34 def mult():
35     inputs = request.get_json()
36     op = MultOp(xin=inputs['xin'], yin=inputs['yin'])
37     op.cal()
38     return op.to_json(), 200
```

```
root@RAINBOW:~/rainbow/asialab/dccc/dccc-lab/rpc_rest/1_rest/8_hello_world/hello_world# uv run python server.py
```

```
* Serving Flask app 'server'
* Debug mode: on
WARNING: This is a development server. Do not use it in a production
* Running on http://127.0.0.1:8081
Press CTRL+C to quit
* Restarting with stat
* Debugger is active!
* Debugger PIN: 846-852-296
```

```
root@RAINBOW: # curl http://localhost:8081/
{
  "message": "Hello World!"
}
root@RAINBOW: # curl http://localhost:8081/chat/Peter?institution=SUSTech
{
  "message": "Hello Peter from SUSTech!"
}
root@RAINBOW: # curl -X POST -H "Content-Type: application/json" -d '{"xin": 1.5,
"yin": 6}' http://localhost:8081/calculator/mult
{
  "result": 9.0,
  "xin": 1.5,
  "yin": 6
}
```



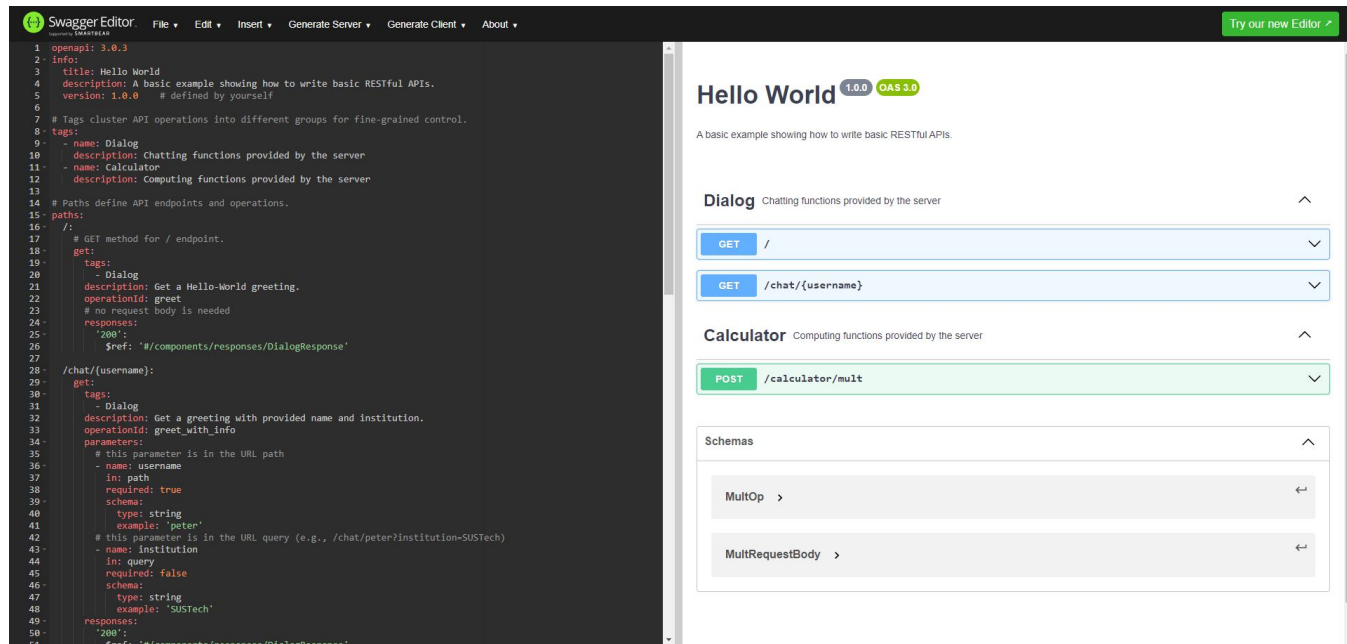
TASK: RESTful API - Hello World with Python Flask

Implement a simple RESTful API server using Python Flask.

> Reference codebase: [rest_hello_world](#)

5. Check the API specification file [hello_world.yaml](#)

6. Copy the file content into the [online Swagger Editor](#). Explore the generated Swagger UI page.



The image shows the Swagger Editor interface. On the left, the YAML specification is displayed in a dark-themed editor. On the right, the generated Swagger UI is shown, featuring a light theme and interactive elements for the API endpoints.

Swagger Editor (Left Panel):

```

1 openapi: 3.0.3
2 info:
3   title: Hello World
4   description: A basic example showing how to write basic RESTful APIs.
5   version: 1.0.0 # defined by yourself
6
7 # Tags cluster API operations into different groups for fine-grained control.
8 tags:
9   - name: Dialog
10     description: Chatting functions provided by the server
11   - name: Calculator
12     description: Computing functions provided by the server
13
14 # Paths define API endpoints and operations.
15 paths:
16   /:
17     # GET method for / endpoint.
18     get:
19       tags:
20         - Dialog
21       description: Get a Hello-World greeting.
22       operationId: greet
23       # no request body is needed
24       responses:
25         '200':
26           $ref: '#/components/responses/DialogResponse'
27
28   /chat/{username}:
29     get:
30       tags:
31         - Dialog
32       description: Get a greeting with provided name and institution.
33       operationId: greet_with_info
34       parameters:
35         # this parameter is in the URL path
36         - name: username
37           in: path
38           required: true
39           schema:
40             type: string
41             example: 'peter'
42         # this parameter is in the URL query (e.g., /chat/peter?institution=SUSTech)
43         - name: institution
44           in: query
45           required: false
46           schema:
47             type: string
48             example: 'SUSTech'
49       responses:
50         '200':
51           $ref: '#/components/responses/DialogResponse'
  
```

Swagger UI (Right Panel):

The UI displays the API title "Hello World" with version "1.0.0" and "OAS 3.0". Below the title, a description states: "A basic example showing how to write basic RESTful APIs."

The UI is organized into sections:

- Dialog** (Chatting functions provided by the server):
 - GET /
 - GET /chat/{username}
- Calculator** (Computing functions provided by the server):
 - POST /calculator/mult
- Schemas**:
 - MultiOp
 - MultiRequestBody

Another RESTful API Example - Petstore


Swagger Editor
File Edit Insert Generate Server Generate Client About

```

1 openapi: 3.0.3
2 info:
3   title: Swagger Petstore
4   description: |-
5     This is a sample Pet S
6     Swagger at [https://sw
7     You can now help us im
8     That way, with time, w
9
10    _If you're looking for the Swagger 2.0/OAS 2.0 version of Petstore, then click [here](https://editor.swagger
11    .io?url=https://petstore.swagger.io/v2/swagger.yaml). Alternatively, you can load via the 'Edit > Load
12    Petstore OAS 2.0' menu option!_
13
14    Some useful links:
15    - [The Pet Store repository](https://github.com/swagger-api/swagger-petstore)
16    - [The source API definition for the Pet Store](https://github.com/swagger-api/swagger-petstore/blob/master
17    /src/main/resources/openapi.yaml)
18
19    termsOfService: http://swagger.io/terms/
20 contact:
21   email: apiteam@swagger.io
22 license:
23   name: Apache 2.0
24   url: http://www.apache.org/licenses/LICENSE-2.0.html
25 version: 1.0.11
26 externalDocs:
27   description: Find out more about Swagger
28   url: http://swagger.io
29 servers:
30   - url: https://petstore3.swagger.io/api/v3
31 tags:
32   - name: pet
33     description: Everything about your Pets
34     externalDocs:
35       description: Find out more
36       url: http://swagger.io
37   - name: store
38     description: Access to Petstore orders
39     externalDocs:
40       description: Find out more about our store
41       url: http://swagger.io
42   - name: user
43     description: Operations about user
44 paths:
45   /pet:
46     put:
47       tags:
48         - pet
49       summary: Update an existing pet
50       description: Update an existing pet by Id
51       operationId: updatePet

```

Convert to YAML
Load Petstore OAS 3.0
Load Petstore OAS 2.0

Swagger Petstore - OpenAPI 3.0 1.0.11 OAS 3.0

This is a sample Pet Store Server based on the OpenAPI 3.0 specification. You can find out more about Swagger at <https://swagger.io>. In the third iteration of the pet store, we've switched to the design first approach! You can now help us improve the API whether it's by making changes to the definition itself or to the code. That way, with time, we can improve the API in general, and expose some of the new features in OAS3.

If you're looking for the Swagger 2.0/OAS 2.0 version of Petstore, then click [here](#). Alternatively, you can load via the [Edit > Load Petstore OAS 2.0](#) menu option!

Some useful links:

- [The Pet Store repository](#)
- [The source API definition for the Pet Store](#)

Terms of service

Contact the developer

Apache 2.0

Find out more about Swagger

Servers

<https://petstore3.swagger.io/api/v3>

[Authorize](#)

pet Everything about your Pets [Find out more](#)

PUT [/pet](#) Update an existing pet

POST [/pet](#) Add a new pet to the store

GET [/pet/findByStatus](#) Finds Pets by status

Another RESTful API Example - Petstore

The image shows the Swagger Editor on the left and the Swagger UI on the right. The Swagger Editor displays the OpenAPI 3.0 specification for a Pet Store API. The Swagger UI shows the endpoint `GET /pet/findByStatus` with the status parameter set to `available`. The `Execute` button is highlighted with a red box, and the `Responses` section is also highlighted with a red box. Red arrows point to the `status` parameter value and the `Execute` button. The response body is shown as a JSON object.

Swagger JSON Definition (Left Pane):

```

openapi: 3.0.3
info:
  title: Swagger Petstore - OpenAPI 3.0
  description: |-
    This is a sample Pet Store Server based on the OpenAPI 3.0 specification. You can find out more about
    Swagger at [https://swagger.io](https://swagger.io) or [https://swagger.io](https://swagger.io). In the third iteration of the pet store, we've switched
    to the design first approach!
    You can now help us improve the API whether it's by making changes to the definition itself or to the code.
    That way, with time, we can improve the API in general, and expose some of the new features in OAS3.
  _If you're looking for the Swagger 2.0/OAS 2.0 version of Petstore, then click [here](https://editor.swagger
  .io/?url=https://petstore.swagger.io/v2/swagger.yaml). Alternatively, you can load via the 'Edit > Load
  Petstore OAS 2.0' menu option!_
  Some useful links:
  - [The Pet Store repository](https://github.com/swagger-api/swagger-petstore)
  - [The source API definition for the Pet Store](https://github.com/swagger-api/swagger-petstore/blob/master
  /src/main/resources/openapi.yaml)
  termsOfService: http://swagger.io/terms/
  contact:
    email: apiteam@swagger.io
  license:
    name: Apache 2.0
    url: http://www.apache.org/licenses/LICENSE-2.0.html
  version: 1.0.11
externalDocs:
  description: Find out more about Swagger
  url: http://swagger.io
servers:
  - url: https://petstore3.swagger.io/api/v3
tags:
  - name: pet
    description: Everything about your Pets
    externalDocs:
      description: Find out more
      url: http://swagger.io
  - name: store
    description: Access to Petstore orders
    externalDocs:
      description: Find out more about our store
      url: http://swagger.io
  - name: user
    description: Operations about user
paths:
  /pet:
    put:
      tags:
        - pet
      summary: Update an existing pet
      description: Update an existing pet by Id
      operationId: updatePet
      requestBody:
        description: Update an existent pet in the store
        content:
          application/json:
            schema:
              $ref: '#/components/schemas/Pet'
          application/xml:
            schema:
              $ref: '#/components/schemas/Pet'

```

Swagger UI (Right Pane):

Endpoint: `GET /pet/findByStatus` Finds Pets by status

Parameters:

Name	Description
status	Status values that need to be considered for filter

Execute: `available`

Responses:

200 Response body

```

{
  "id": 7759440800036640000,
  "category": {
    "id": 3390426472185137000,
    "name": "MAVRZVL'yOB'ZcUY6-{FNO-VJ2}3uF;ar3ER_x+y))('`
  },
  "name": "y27M1B0as5vAN(N''*o=H[1g02;0L-vy,5{1,1'",
  "photoUrls": [
    "Lzm3j'KYpF'/k0t[1m03S'b>60tc'u)-X,t-DF;KL3[(-<X)P]]('`K"
  ],
  "tags": [

```

RESTful API Components

Take OpenAPI specification as an example:

- Resource: identified by a URL endpoint.
- HTTP method: specifies an operation on the resource (e.g., GET, POST, etc.).
- HTTP request:
 - Header: metadata
 - Content-Type = application/json
 - Query Parameters
 - /users?age=25&sort=desc
 - Request Body
- HTTP response:
 - Status code + message
 - 200 OK
 - 404 Not Found
 - ...
 - Header
 - Response Body

Request URL

`https://petstore3.swagger.io/api/v3/pet`

POST /pet Add a new pet to the store

Add a new pet to the store

Parameters

No parameters

Request body required application/json

Create a new pet in the store

Example Value | Schema

```
{
  "id": 10,
  "name": "doggie",
  "category": {
    "id": 1,
    "name": "Dogs"
  },
  "photoUrls": [
    "string"
  ],
  "tags": [
    {
      "id": 0,
      "name": "string"
    }
  ],
  "status": "available"
}
```

Responses

Code	Description	Links
200	Successful operation	No links

Media type: application/json

Example Value | Schema

Monolithic Service ➡ Microservices



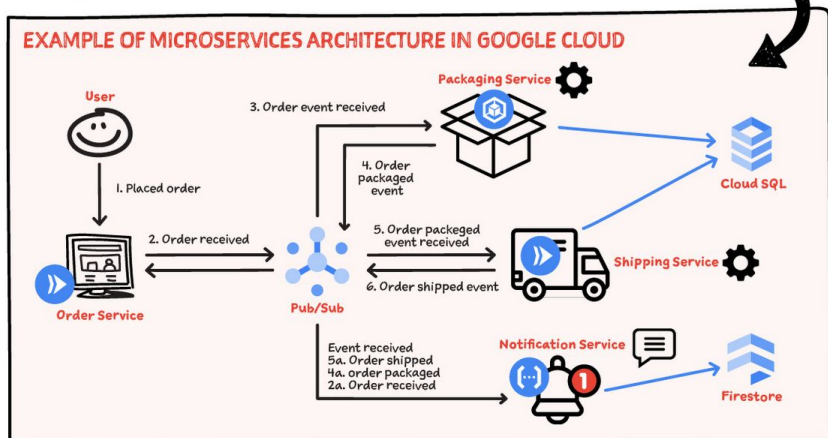
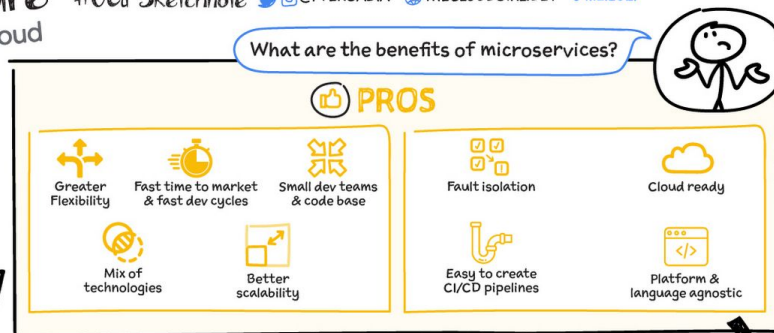
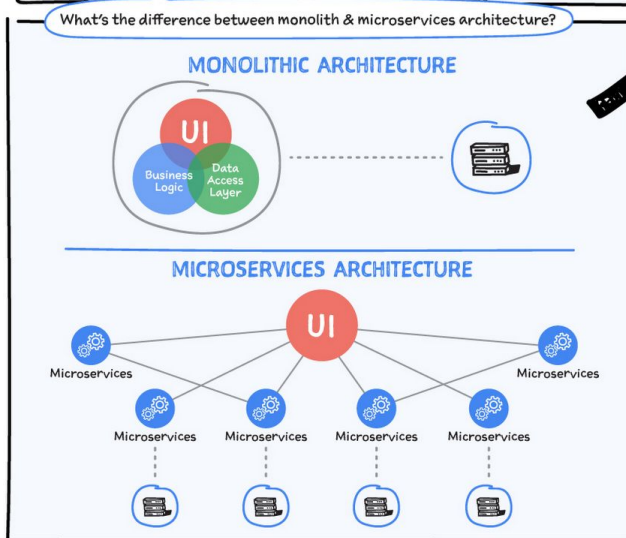
Microservices Architecture
on Google Cloud

#GCPsketchnote

@PVERGADIA

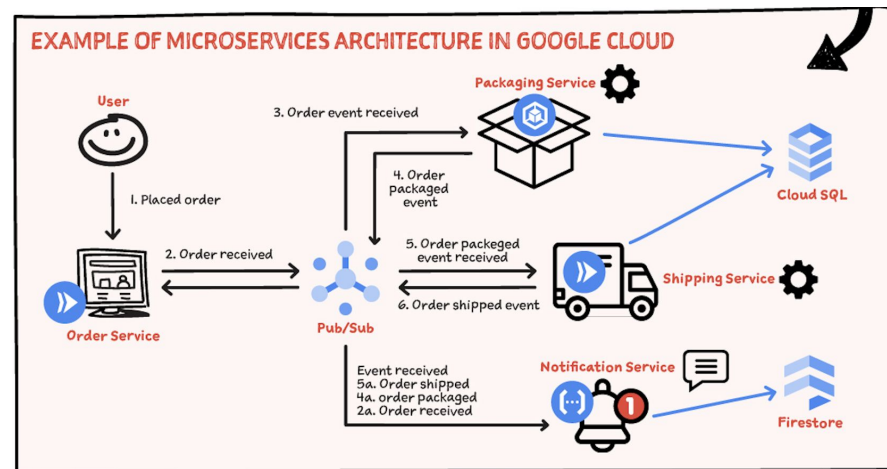
THECLOUDGIRL.DEV

04.12.2021



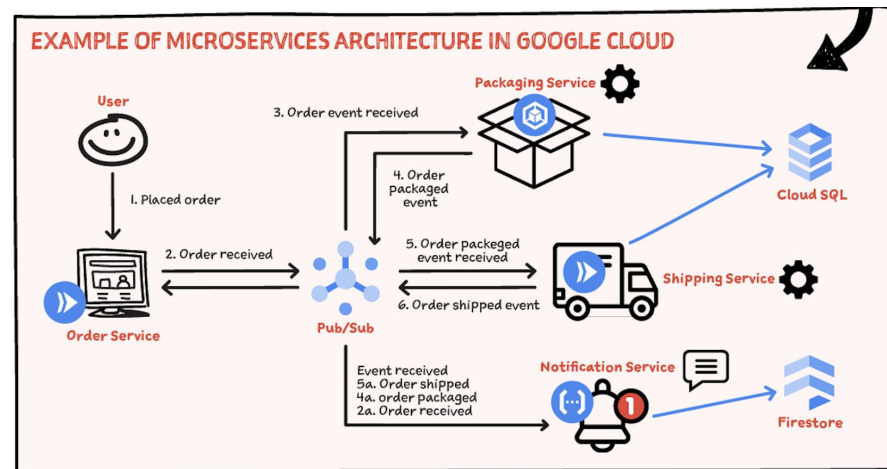
Microservices

- Definition: “an architectural pattern that arranges an application as a collection of loosely coupled, fine-grained services, communicating through lightweight protocols.”
- Features:
 - a. Finer granularity: each microservice is small and modular
 - b. Improved Flexibility & Scalability: easier migration & load balancing
 - c. Loosely coupled:
 - Design phase: refactoring a microservice does not heavily affect the others.
 - Deployment phase: self-contained microservices can be deployed concurrently.
 - Runtime phase: an unavailable microservice does not severely affect the others.



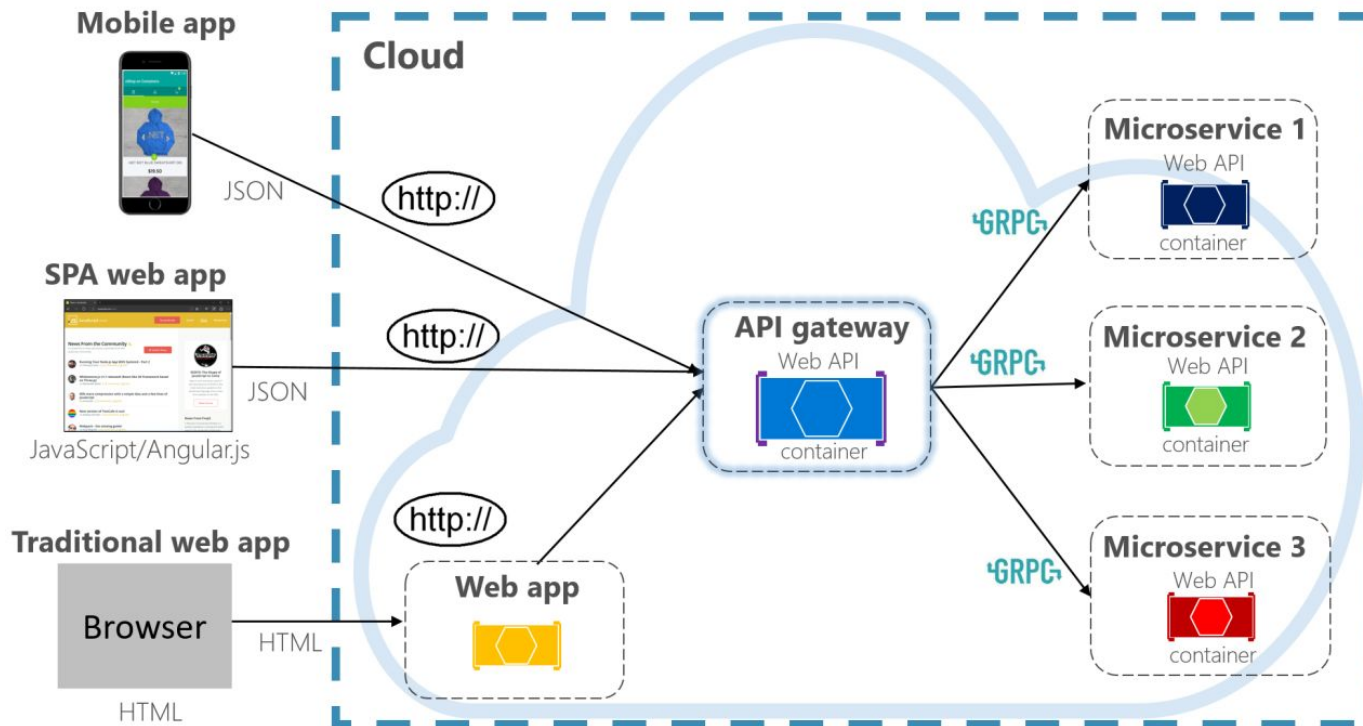
Microservices

- Definition: “an architectural pattern that arranges an application as a collection of loosely coupled, fine-grained services, communicating through lightweight protocols.”
- Features:
 - a. Finer granularity: each microservice is small and modular
 - b. Improved Flexibility & Scalability: easier migration & load balancing
 - c. Loosely coupled
 - d. Cross-platform & Cross-Language: each microservice can be written with different programming languages and deployed to different OS platforms.
 - e. Cloud Ready: easier packaging into containers, more efficient resource utilization



- **gRPC** is currently a popular architecture for microservices!

Microservices with gRPC



TASK: gRPC - Hello World

Implement a Hello World gRPC Python example.

> Reference codebase: `rpc_grpc_hello_world`

1. Set up Python ([uv](#) is recommended).
2. Install Python dependencies into a uv environment via:
 - `uv sync`
3. Check the protocol file `assistant.proto`. Use `protoc` to generate some code:
 - `uv run python -m grpc_tools.protoc -I./ --python_out=. --pyi_out=. --grpc_python_out=. assistant.proto`
 - These generated code will be utilized to implement the gRPC client and the gRPC server.

```

assistant.proto X
0_rpc > 0_rpc > 0_hello_world > 0 assistant.proto > ...
1 syntax = "proto3";
2
3 // Style Guide: https://protobuf.dev/programming-guides/style/.
4 // Files should be named "lower_snake_case.proto".
5
6 /*
7 Services are what the servers provide for the clients. Specifically for gRPC.
8 Use PascalCase (with an initial capital) for both the service name and any RPC method names.
9 */
10 service AssistantService {
11   // Constructs a greeting message based on the given information of the user.
12   rpc GreetWithInfo(GreetRequest) returns (GreetResponse);
13   // Multiply two given numbers and give back the output (with the inputs).
14   rpc Multiply(MultRequest) returns (MultResponse);
15 }
16
17 /*
18 Messages are exchanged between clients and servers.
19 Use PascalCase (with an initial capital) for message names: SongServerRequest.
20 Prefer to capitalize abbreviations as single words: GetDnsRequest rather than GetDNSRequest.
21 Use lower_snake_case for field names, including oneof field and extension names: song_name.
22 */
23 // The greeting request message with the user's name and institution.
24 message GreetRequest {
25   string user_name = 1; // user's name at the 1st position
26   string institution = 2; // user's institution at the 2nd position
27 }
28
29 // The greeting response message with the constructed message.
30 message GreetResponse {
31   string message = 1;
32 }
33
34 // The multiplication request message including two input double numbers.
35 message MultRequest {
36   double xin = 1;
37   double yin = 2;
38 }
39
40 // The multiplication response message including the inputs and the output number.
41 message MultResponse {
42   double xin = 1;
43   double yin = 2;
44   double result = 3;
45 }
46

```

```

ls -lh
Makefile
README.md
assistant.proto
assistant_pb2.py
assistant_pb2.pyi
assistant_pb2_grpc.py
hw_client.py
hw_server.py
pyproject.toml
requirements.txt
uv.lock

```

protoc

TASK: gRPC - Hello World

Implement a Hello World gRPC Python example.

> Reference codebase: `rpc_grpc_hello_world`

1. Set up Python ([uv](#) is recommended).
2. Install Python dependencies into a uv environment via:
 - `uv sync`
3. Use `protoc` to generate some code.
4. Implement the gRPC server in `hw_server.py`. Run the gRPC server via:
 - `uv run python hw_server.py`
5. Implement the gRPC client in `hw_client.py`. In another terminal, run the gRPC client via:
 - `uv run python hw_client.py`

```
root@RAINBOW: /rainbow/ai/ml/microservices/rpc/grpc/hello_world/hello_world# uv run python hw_server.py
INFO:root:Server started, listening on 8082
```

```
root@RAINBOW: /rainbow/ai/ml/microservices/rpc/grpc/hello_world/hello_world# uv run python hw_client.py
> Greet: Hello Assistant?
> Client received:
message: "Hello Peter from SUSTech!"

> Mult: Requesting a multiplication task
> Client received:
xin: 3.5
yin: 5
result: 17.5
```

```
hw_server.py
0_rpc > 0_grpc > 0_hello_world > hw_server.py > ...
1 from concurrent import futures
2 import logging
3
4 import grpc
5 from assistant_pb2_grpc import AssistantServiceServicer, add_AssistantServiceServicer_to_server
6 from assistant_pb2 import GreetRequest, GreetResponse, MultRequest, MultResponse
7
8 class Assistant(AssistantServiceServicer):
9     def GreetWithInfo(self, request: GreetRequest, context):
10         msg = f'Hello {request.user_name} from {request.institution}!'
11         return GreetResponse(message=msg)
12
13     def Multiply(self, request: MultRequest, context):
14         res = request.xin * request.yin
15         return MultResponse(xin=request.xin, yin=request.yin, result=res)
16
17 def serve():
18     port = '8082'
19     # the server can handle 10 client requests concurrently
20     server = grpc.server(futures.ThreadPoolExecutor(max_workers=10))
21     add_AssistantServiceServicer_to_server(Assistant(), server)
22     # [:::] specifies the listen on all ipv4/ipv6 addresses
23     server.add_insecure_port('[:::]' + port)
24     server.start()
25     logging.info(f'Server started, listening on {port}')
26     server.wait_for_termination()
27
28 if __name__ == '__main__':
29     logging.basicConfig(level=logging.INFO)
30     serve()
31
```

“abstracts the complexity of network communication”

```
hw_client.py
0_rpc > 0_grpc > 0_hello_world > hw_client.py > ...
1 import grpc
2 from assistant_pb2 import GreetRequest, MultRequest
3 from assistant_pb2_grpc import AssistantServiceStub
4
5 def run():
6     with grpc.insecure_channel('localhost:8082') as channel:
7         stub = AssistantServiceStub(channel)
8         # Greeting
9         print('> Greet: Hello Assistant?')
10        res = stub.GreetWithInfo(GreetRequest(user_name='Peter', institution='SUSTech'))
11        print(f'> Client received:\n{res}')
12        # Multiplication
13        print('> Mult: Requesting a multiplication task')
14        res = stub.Multiply(MultRequest(xin=3.5, yin=5))
15        print(f'> Client received:\n{res}')
16
17 if __name__ == '__main__':
18     run()
19
```


TASK: gRPC - Quick Start with Go

Hello World again, but this time with Go.

We will also use a bit of [Go](#) in our future lab sessions to demonstrate the cross-language feature of gRPC. Set up Go and try the [official quick start guide](#).

```
root@RAINBOW:~# go version
go version go1.25.1 linux/amd64
```

1. Install Go according to the [official documentation](#). Verify with `go version`.
2. (optional) Set a proxy for Go to smoothly download the dependencies:
 - `export GOPROXY=https://goproxy.cn,direct`
3. Install the Protobuf compiler, `protoc`, version 3 according to the [official documentation](#).
4. Install Go plugins for Protobuf:
 - `go install google.golang.org/protobuf/cmd/protoc-gen-go@latest`
 - `go install google.golang.org/grpc/cmd/protoc-gen-go-grpc@latest`
 - These plugins will be installed to a `bin/` folder from a preset `GOPATH`. Add this folder to the system path and `protoc` will be able to find these plugins:

```
root@RAINBOW:~# go env GOPATH
/root/go
```

 - `export PATH="$PATH:$(go env GOPATH)/bin"`
5. Clone the official gRPC Go GitHub repository and enter the Hello World example folder:
 - `git clone -b v1.75.0 --depth 1 https://github.com/grpc/grpc-go`
 - `cd grpc-go/examples/helloworld`

TASK: gRPC - Quick Start with Go

Hello World again, but this time with Go.

We will also use a bit of [Go](#) in our future lab sessions to demonstrate the cross-language feature of gRPC. Set up Go and try the [official quick start guide](#).

6. Check the proto file:
`helloworld/helloworld.proto`. Some Go code has already been generated by protoc within the same directory. To keep it simple, we will not generate them again for this lab session.
7. Run the server code:
 - `go run greeter_server/main.go`
8. In another terminal, run the client code:
 - `go run greeter_client/main.go`

```
(base) root@RAINBOW: /home/raibow/learn/grpc-go/examples/helloworld# go run greeter_server/main.go
go: downloading google.golang.org/genproto v0.0.0-20240814211410-ddb44dafa142
2024/10/09 06:22:37 server listening at [::]:50051
```

```
(base) root@RAINBOW: /home/raibow/learn/grpc-go/examples/helloworld# go run greeter_client/main.go
2024/10/09 06:30:34 Greeting: Hello world
```

```
helloworld.proto x
helloworld > helloworld.proto > ...

15 syntax = "proto3";
16
17 option go_package = "google.golang.org/grpc/examples/helloworld/helloworld";
18 option java_multiple_files = true;
19 option java_package = "io.grpc.examples.helloworld";
20 option java_outer_classname = "HelloWorldProto";
21
22 package helloworld;
23
24 // The greeting service definition.
25 service Greeter {
26   // Sends a greeting
27   rpc SayHello (HelloRequest) returns (HelloReply) {}
28 }
29
30 // The request message containing the user's name.
31 message HelloRequest {
32   string name = 1;
33 }
34
35 // The response message containing the greetings
36 message HelloReply {
37   string message = 1;
38 }
```

TASK: gRPC - Quick Start with Go

Hello World again, but this time with Go.

We will also use a bit of [Go](#) in our future lab sessions to demonstrate the cross-language feature of gRPC. Set up Go and try the [official quick start guide](#).

6. Check the proto file:
[helloworld/helloworld.proto](#). Some Go code has already been generated by protoc within the same directory. To keep it simple, we will not generate them again for this lab session.
7. Run the server code:
 - `go run greeter_server/main.go`
8. In another terminal, run the client code:
 - `go run greeter_client/main.go`

```
(base) root@RAINBOW: ~/go/src/github.com/GoogleCloudPlatform/grpc-gcp-go/examples/helloworld # go run greeter_client/main.go
2024/10/09 06:27:57 could not greet: rpc error: code = DeadlineExceeded desc = context deadline excee
ded
exit status 1
```

Q: getting DeadlineExceeded?

```
main.go  X
greeter_client > main.go > ...
19 // Package main implements a client for Greeter service.
20 package main
21
22 import (
23     "context"
24     "flag"
25     "log"
26     "time"
27
28     "google.golang.org/grpc"
29     "google.golang.org/grpc/credentials/insecure"
30     pb "google.golang.org/grpc/examples/helloworld/helloworld"
31 )
32
33 const (
34     defaultName = "world"
35 )
36
37 var (
38     addr = flag.String("addr", "127.0.0.1:50051", "the address to connect to")
39     name = flag.String("name", defaultName, "Name to greet")
40 )
41
42 func main() {
43     flag.Parse()
44     // Set up a connection to the server.
45     conn, err := grpc.NewClient(*addr, grpc.WithTransportCredentials(insecure.NewCredentials()))
46     if err != nil {
47         log.Fatalf("did not connect: %v", err)
48     }
49     defer conn.Close()
50     c := pb.NewGreeterClient(conn)
51
52     // Contact the server and print out its response.
53     ctx, cancel := context.WithTimeout(context.Background(), 15*time.Second)
54     defer cancel()
55     r, err := c.SayHello(ctx, &pb.HelloRequest{Name: *name})
56     if err != nil {
57         log.Fatalf("could not greet: %v", err)
58     }
59     log.Printf("Greeting: %s", r.GetMessage())
60 }
61
```

Summary

- Distributed Architecture Patterns
 - a. Web Services
 - Serve clients over the Internet using web standard protocols & data formats.
 - **RESTful API** is currently the most popular architecture for Web Services.
 - b. Microservices
 - Break services into loosely coupled, fine-grained services, communicating through lightweight protocols
 - **gRPC** is a popular architecture for microservices.
- 6 Most Popular API Architecture Patterns
 - a. SOAP
 - b. RESTful API**
 - c. GraphQL
 - d. RPC**
 - e. WebSocket
 - f. Webhook

More details in the following lab sessions...

